# BitSemCom + GRAND

A guide to the repository: what is where, how it works, and how to run it

Safaa Salman

*Thesis working document*, November 2026, branch `grand-implementation`

**What this document is**

This document explains, in plain language, what is in the `BitSemCom` repository, how one image travels through the system, what every folder and script does, where the results are, and which conventions you must know before touching the code. It is written for students who join the project. The step-by-step commands are in the `README.md` at the repository root; this guide explains what those commands do. The numbers we measured are in the companion document *BitSemCom + GRAND: results so far*.

## Contents

**1 What this repository is** **2**

**2 How one image travels through the system** **2**

**3 Repository map** **3**

**4 Training: without and with GRAND** **4**

4.1 The original recipe (`main.py`) . . . 4

4.2 Our trainer (`train_bitsemcom.py`) . . . 4

4.3 How GRAND is put inside the loop . . . 5

**5 Evaluation** **5**

5.1 Receivers . . . 5

5.2 Channel conditions . . . 5

5.3 The two sweep runners . . . 6

5.4 Metrics . . . 6

5.5 Figures . . . 6

**6 The GRAND package** **6**

**7 Data, checkpoints and results** **7**

**8 Conventions you must know** **7**

**9 The most useful commands** **7**

**10 Glossary** **8**

# 1 What this repository is

The repository has two layers.

**The original BitSemCom.** Bit-level semantic image communication from Shanghai Jiao Tong University (arXiv 2510.26225). A Swin-transformer autoencoder (the "WITT" backbone) turns an image into a short sequence of bits, the bits are sent over a noisy channel as QPSK symbols, and a second Swin network rebuilds the image from what arrives. The authors call it *semantic* because the network decides what to put in the bits; nothing in the bitstream is a pixel value. This code is in `main.py`, `network/`, `data/`, `loss/` and `utils.py`, untouched.

**Our addition: a channel code decoded by GRAND.** The thesis question is whether an explicit error-correcting code, decoded with GRAND (Guessing Random Additive Noise Decoding), improves the image quality of such a system. We put a CRC code between the bit mapper and the semantic decoder, decode it with ORBGRAND (the soft-input version of GRAND), and measure image quality with and without it, on several channels, with several receivers, and also with GRAND inside the training loop. That code is in `grand/`, `experiments/`, `train_bitsemcom.py` and `matlab/`.

Branch `main` holds the original code only; branch `grand-implementation` holds everything described here.

# 2 How one image travels through the system

Figure 1 shows the path of one $256 \times 256$ image. The grey boxes are the original BitSemCom, the orange boxes are what we added. Each box is explained below.

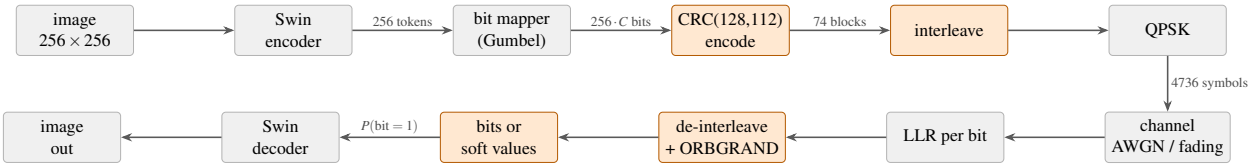



Figure 1: One image through the system for $C = 32$. Grey: the original BitSemCom. Orange: the coded link we added. Without the orange boxes the LLRs are turned into $P(\text{bit} = 1)$ directly and handed to the decoder; that is the original receiver.

**Encoder and tokens.**
The Swin encoder downsamples the image by 16 in each direction, so a $256 \times 256$ image becomes $16 \times 16 = 256$ *tokens*, each a feature vector.

**Bit mapper and $C$.**
For every token the bit mapper produces $C$ bits with a Gumbel-softmax layer (`Resample_Layer` in `network/network.py`). $C$ is the only size knob: $C = 16$, 32, 64 give 4096, 8192, 16384 bits per image. The layer is stochastic: sampling it twice flips about 5% of the bits, and the decoder was trained to live with that.

**CRC code (ours).**
The bits are cut into blocks of $k = 112$ and each block gets 16 parity bits from a CRC polynomial (Koopman notation `0x9eb2`), giving codewords of $n = 128$ bits, rate 0.875. The code is *systematic*: the codeword is the 112 data bits followed by the 16 parity bits. For $C = 32$ the 8192 bits are padded to 74 blocks (9472 coded bits), 16% more than uncoded.

**Interleaver (ours).**
A fixed pseudo-random permutation of all coded bits, known to both sides. In fading, a deep fade wipes out a whole token; after interleaving its bits are spread over many codewords and many tokens, which both the code and the semantic decoder handle far better.

**QPSK and channel.**
Two bits per symbol, Gray mapping, exactly the mapping of the original `QPSK_soft`. Channels: AWGN, or block Rayleigh fading with one complex gain per block of symbols (`fade8` = one gain per 8 symbols = one $C = 16$ token). Optional channel-state errors: the receiver may assume an SNR that is wrong by $\pm 3$ dB, or use a noisy channel estimate.

**LLR.**
For every bit the receiver computes a log-likelihood ratio. Convention everywhere in this repository and in Duffy's code: **positive LLR means bit 0**. The original receiver converts it to $P(\text{bit}=1)=\sigma(-\text{LLR})$ and gives that number to the decoder.

**ORBGRAND (ours).**
GRAND does not decode the code directly; it guesses the *noise*. It sorts the bits by reliability $|\text{LLR}|$, flips the least reliable bits first in a fixed order of patterns (the "landslide" pattern generator), and after each guess checks whether the result is a valid codeword ($Hc=0$). The first valid codeword wins. If no codeword is found within the *guess budget* the block is *abandoned*. If a wrong codeword passes the check the block is *miscorrected*: the decoder then receives confidently wrong bits, which is the one way GRAND can hurt.

**What the decoder receives (ours).**
Decoded blocks: exact 0/1 values of their 112 data bits. Abandoned blocks: the plain soft values the uncoded receiver would have used (*soft fallback*; *hard fallback* gives 0/1 decisions instead). The padding is dropped and the $256 \cdot C$ values go to the Swin decoder, which cannot tell which receiver produced them.

# 3 Repository map

| Path | What it is |
|---|---|
| `main.py` | the original training and test script (two stages: error-free channel, then noisy channel). Hard-coded dataset paths in its `config` class. Needs `lpips`, `thop`, `tensorboardX` and a CUDA GPU. |
| `network/network.py` | the `WITT` model: encoder, bit mapper (`Resample_Layer`), `QPSK_soft` (channel + LLR + $P(1)$), decoder. |
| `network/encoder.py`, `decoder.py`, `modules.py` | Swin blocks. `network/channel.py` is unused by the bit path. |
| `data/datasets.py`, `loss/distortion.py`, `utils.py` | loaders, MS-SSIM, logging helpers of the original code. |
| `train_bitsemcom.py` | our trainer: fixed step budget, warmup + cosine learning rate, random SNR per batch, validation every $N$ steps, best checkpoint, resume; `--grand` puts the coded link + ORBGRAND into the loop. |
| `grand/codes.py` | builds CRC and random linear codes: generator matrix $G=[I\,\vert\,P]$, parity-check matrix $H=[P^T\,\vert\,I]$, `encode`, `is_codeword`. |
| `grand/orbgrand.py` | ORBGRAND in pure numpy (basic and 1-line variants, landslide generator), port of Duffy's MATLAB. `ORBGRAND(H, max_queries).decode(llr)` returns (word, guesses, abandoned). |
| `grand/selftest.py`, `grand/validate_duffy.py` | self-test; reproduction of the block error rate curves shipped with GRAND-MATLAB. |
| `experiments/bitsemcom_io.py` | builds the model without importing `main.py`; loads checkpoints with a strict key check; encodes images to bits and decodes probabilities to images. |
| `experiments/channel_coding.py` | the coded link (`CodedLink.transmit`): pad, encode, interleave, QPSK, channel with CSI errors, LLR, ORBGRAND, fallback, statistics. Also the uncoded QPSK channel in numpy. |
| `experiments/torch_channel.py` | the same uncoded QPSK channel in torch, differentiable, used in training. |
| `experiments/grand_layer.py` | `GrandChannel`: GRAND inside training. Forward value = the coded link's output; gradient = the uncoded soft channel's Jacobian (section 4). |
| `experiments/conditions.py` | the grammar of channel-condition names (section 5.2). |
| `experiments/metrics.py` | PSNR, MS-SSIM, LPIPS (AlexNet), CLIP ViT-B/32 cosine similarity. |

| Path | What it is |
|---|---|
| `experiments/run_grand.py` | round-1 sweep runner: one model, several SNRs, all receivers; one JSONL row per (image, SNR, receiver). |
| `experiments/evaluate.py` | round-2 sweep runner: several models × conditions × SNRs × receivers, four metrics, optional MATLAB decoding, example images. |
| `experiments/plot_llr.py` | the LLR study: what the decoder and GRAND receive under each condition. |
| `experiments/plot_results.py`, `plot_training.py`, `plot_review_gains.py`, `plot_style.py` | all figures: quality curves, coding gains, GRAND block statistics, training curves, the paired-gain charts of the review, example strips; one shared style. |
| `experiments/summarize.py` | markdown tables from round-1 JSONLs. |
| `experiments/bit_importance.py`, `channel_importance.py` | round-1 diagnostics: flip a fraction of bits (random, most confident, least confident, whole tokens, per channel) and measure the PSNR drop. |
| `experiments/grand_verify.py`, `grand_backend.py`, `matlab/` | export of a test set of LLR blocks, bit-exact comparison with Duffy's MATLAB code, and a decoding backend that routes every ORBGRAND call through MATLAB or Octave. |
| `experiments/make_kodak_patches.py` | makes the 24 centre 256 × 256 crops of Kodak used everywhere. |
| `BitSemCom_Colab.ipynb` | the round-2 experiments as Colab cells (models, data and results on Google Drive). |
| `results/` | raw results, tables, figures, training logs (section 7). |
| `docs/` | the two markdown reports, their figures, and these LaTeX documents. |
| `Pretrain/`, `datasets/` | checkpoints and images; git-ignored, see the README for where to get them. |

# 4 Training: without and with GRAND

## 4.1 The original recipe (`main.py`)

The authors train in two stages. Stage 1 trains the autoencoder with an error-free channel (the `WITT_BitSC_C*` checkpoints). Stage 2 starts from stage 1 and fine-tunes with `--pass_channel`: each epoch draws one integer SNR between `--snr_min` and `--snr_max`, sends the bits through `QPSK_soft` and trains the whole network on the reconstruction MSE. The learning rate is stepped down by `--decay` every `--step` epochs to `--min_lr`. The loop never ends by itself (ten million epochs); a checkpoint is written every 20 epochs to `history/BitSC_Prob/<timestamp>/models/`. Our starting checkpoints `Pretrain/BitSemCom_C{16,32,64}_stage2_ep*.model` are stage-2 models trained this way on Colab, with SNR in $[-3, 6]$ dB.

## 4.2 Our trainer (`train_bitsemcom.py`)

The supervisor asked for a benchmark "trained properly": a fixed budget of 30,000 steps, AdamW, 500 warmup steps then cosine decay from $10^{-4}$ to $10^{-6}$, gradient clipping, SNR drawn per batch from the continuous range $[0, 10]$ dB, bf16 autocast for the Swin blocks, validation every 1000 steps on the 24 Kodak crops at 0, 3, 6 and 10 dB with fixed seeds, and the best checkpoint kept. One training step is:

1. image → encoder → bit mapper → bits (the Gumbel layer passes a gradient through its own straight-through trick);
2. channel: either the uncoded torch QPSK (`channel_uncoded`) or, when `--grand` is set, the GRAND channel described in the next subsection;
3. $P(1)$ → decoder → MSE on the 0–255 scale → backward → AdamW step.

### 4.3 How GRAND is put inside the loop

`GrandChannel.transmit` (`experiments/grand_layer.py`) moves the batch's bits to numpy, sends each image to a worker process, and the worker runs the very same `CodedLink.transmit` that evaluation uses (CRC encode, interleave, QPSK, channel, LLR, ORBGRAND with budget 2000, soft fallback). The $P(1)$ values come back as a tensor of the same shape the uncoded channel would return, so the decoder cannot tell which channel it is attached to.

**Why the gradient needs a trick**

Encoding, interleaving and ORBGRAND are not differentiable. The trainer therefore pushes the same bits through the uncoded torch channel a second time (fresh noise, same settings) and combines the two:

$$p_1 \;=\; \mathrm{stopgrad}(p_1^{\mathrm{GRAND}}) \;+\; \big(p_1^{\mathrm{soft}} - \mathrm{stopgrad}(p_1^{\mathrm{soft}})\big).$$

The value is exactly the GRAND output (the bracket is zero) while the gradient is the uncoded soft channel's Jacobian, so the encoder is trained by the same gradient path as in the benchmark run; only the forward signal the decoder sees differs (`--grand_grad soft`, the default; `identity` is plain straight-through).

The block statistics of every step (abandoned, miscorrected, guesses, decode time) go to `log.csv`. In the run we made, about half of the blocks were abandoned and 1.5% miscorrected, flat for all 30k steps, because with fixed-power QPSK the encoder has no handle on which blocks decode.

## 5 Evaluation

### 5.1 Receivers

Every receiver gets the same semantic bits and the same channel noise for a given (image, SNR, seed), so all comparisons are paired: the difference between two receivers on one image is a difference between two runs that share everything except the receiver.

| Name | What the semantic decoder receives |
|---|---|
| `noiseless` | the exact bits: the ceiling for any channel code |
| `uncoded_soft` | $P(\text{bit}=1)$ from the QPSK soft demodulator: the original BitSemCom receiver |
| `uncoded_soft_il` | the same, with the coded link's interleaver but no code: isolates the interleaving gain |
| `uncoded_hard` | hard 0/1 decisions |
| `coded_soft` | interleaver + CRC(128,112) + ORBGRAND, abandoned blocks fall back to the soft values |
| `coded_hard` | the same, abandoned blocks fall back to hard decisions |
| `*_ebn0`, `*_noil` | round 1 only: coded path at equal energy per semantic bit; coded path without interleaver |

### 5.2 Channel conditions

Condition names (`experiments/conditions.py`) are tokens joined by underscores: `awgn` or `fade<N>` ($N$ symbols per fading block; `fade1` is fast fading, `fade8` one gain per 8 symbols); `over<X>` / `under<X>`: the receiver assumes an SNR $X$ dB too high / too low; `h<Y>`: channel-estimate error of variance $Y$ (fading only); `rnd<X>`: SNR error drawn uniformly in $[-X, X]$ per transmission. The round-2 sweep used `awgn`, `awgn_over3`, `awgn_under3`, `fade8`, `fade8_h0.1` at $E_s/N_0$ from 0 to 12 dB in 2 dB steps.

### 5.3 The two sweep runners

`experiments/run_grand.py` (round 1) takes one model and one channel and writes one JSONL row per (image, SNR, receiver) with PSNR, MS-SSIM and, for coded receivers, the block statistics. `experiments/evaluate.py` (round 2) takes several models (`--ckpt name=file`), a list of condition names, SNRs and receivers, adds LPIPS and CLIP, can save example reconstructions (`--save_images`), and can decode through MATLAB (`--grand_backend`). The guess budget (`--budget`) matters: the main sweep uses 2000, the budget the GRAND model was trained with; a second sweep uses 100,000.

Each JSONL row has the keys `model`, `image`, `C`, `snr`, `condition`, `channel`, `fade_block`, `snr_err_db`, `h_err_var`, `path`, `channel_snr`, `payload_bits`, `psnr`, `msssim`, `lpips`, `clip` and, for coded receivers, `n`, `k`, `blocks`, `coded_bits`, `symbols`, `ber_channel`, `ber_after`, `bler_channel`, `bler_after`, `abandon_rate`, `miscorrect_rate`, `avg_queries`, `max_queries_seen`, `decode_time_s`.

### 5.4 Metrics

PSNR is the headline number (mean over the 24 images; differences below about 0.03 dB are noise). MS-SSIM and LPIPS (AlexNet; lower is better) are computed for every image. CLIP similarity (ViT-B/32 cosine similarity between the two images) was meant as a "semantic" metric but is too noisy on 24 images to separate receivers.

### 5.5 Figures

`plot_results.py` draws, per condition, the four metrics against SNR for every model and receiver (`quality_*.png`), the coding gain against the plain receiver (`gain_*.png`) and against the interleaver-only receiver (`gain_*_vs_il.png`), and the GRAND block statistics (`grand_*.png`); it also writes `summary.md`. `plot_training.py` draws the training curves from `log.csv`/`val.csv`. `plot_review_gains.py` draws the paired-gain bar charts of the review (room up to perfect bits against the coded gain, GRAND-trained minus benchmark, cost of imperfect CSI, budget 2000 against 100,000, example reconstruction strips).

## 6 The GRAND package

`grand/codes.py`: `make_crc(k, poly)` builds the systematic generator matrix $G$ ($k \times n$) and parity-check matrix $H$ ($(n-k) \times n$) from a CRC polynomial; `make_rlc` builds a random linear code; `encode(u, G)` is $uG$ mod 2. `grand/orbgrand.py`: `ORBGRAND(H, max_queries, variant)` with `decode(llr)` and `decode_many`. Syndromes are packed into 64-bit integers and the noise patterns are cached, so the decoder runs at roughly two million guesses per second in pure numpy. `budget_mode='level'` reproduces the per-weight-level budget check of Duffy's MATLAB code, so a block may use up to about 15% more guesses than the nominal budget; this mode is used everywhere numbers are compared with the reference.

The port was validated twice: `grand/validate_duffy.py` reproduces the block error rates and average guess counts of the `RESULTS/*.mat` files shipped with GRAND-MATLAB within Monte Carlo noise; and `experiments/grand_verify.py` exports 740 real LLR blocks for a bit-exact comparison with the unmodified MATLAB functions (`matlab/verify_port.m`).

**Not yet verified bit for bit**

The second check has not yet been run with a real MATLAB installation. The Octave attempt stopped on `cumsum(d,'reverse')`, which Octave lacks; `matlab/octave_compat/cumsum.m` is an untested stand-in. Until this is done, every number in the reports comes from the Python port.

# 7 Data, checkpoints and results

**Data.** Kodak (24 images) at `datasets/kodak_full`, and the centre $256 \times 256$ crops at `datasets/kodak_patches256` made by `experiments/make_kodak_patches.py`; every reported number is on those 24 crops. DIV2K train HR (800 images) is used for training only.

**Checkpoints** (`Pretrain/`, not in git): the authors' stage-1 models; the stage-2 models that all experiments start from, named `BitSemCom_C<C>_stage2_ep*.model` for $C = 16, 32, 64$; and the round-2 models `runs/C32_bench/best.model` and `runs/C32_grand/best.model`.

**Results** (`results/`): round-1 JSONLs `C16_*`, `C32_*`, `C64_*` with their markdown tables `summary_*.md` and the diagnostics `bit_importance_*.md`, `channel_importance_*.md`; round-2 sweeps `eval_C32.jsonl` (12,600 rows, budget 2000) and `eval_C32_b1e5.jsonl` (budget 100,000); training logs in `train_logs/` (`log.csv`, `val.csv` and the console output of the benchmark run, the GRAND run and the abandoned first GRAND attempt); all figures in `figures/`; the MATLAB test-set description in `grand_verify/`. Example reconstructions (`results/images_C32/`) and the test set itself (`testset.mat`) are git-ignored and live on the project Drive.

**Reports** (`docs/`): `grand_results.md` (round 1) and `grand_review.md` (round 2, with its figures in `docs/figures/grand_review/`), plus this guide and the results document in `docs/latex/`.

# 8 Conventions you must know

**Conventions used everywhere in the code**

- **LLR sign.** Positive LLR means bit 0, in BitSemCom, in Duffy's code and in ours. The decoder consumes $P(\text{bit} = 1) = \sigma(-\text{LLR})$, never bits and never LLRs.
- **SNR.** Always $E_s/N_0$ per QPSK symbol in dB. The raw bit error rate is $Q(\sqrt{E_s/N_0})$: 16% at 0 dB, 5.8% at 4 dB, 2.3% at 6 dB, 0.6% at 8 dB. Duffy's BPSK curves at $E_b/N_0 = x$ correspond to this code's QPSK at $E_s/N_0 = x + 3$ dB plus the rate term.
- **Equal** $E_b/N_0$ for a rate-$R$ code means running the coded path at $E_s/N_0 + 10\log_{10} R$, i.e. $-0.58$ dB for 112/128.
- **Pairing.** The same (image, SNR, seed) gives the same bits and the same noise for every receiver; report paired differences with their standard error over the 24 images.
- **Budgets.** Evaluate a GRAND-trained model at the budget it was trained with (2000); a larger budget turns abandonments into miscorrections.
- **Code specs** are strings: `crc:0x9eb2:128:112` (CRC polynomial, $n$, $k$), `rlc:128:104:0` (random linear code, $n$, $k$, seed).

# 9 The most useful commands

All commands, with the data and checkpoint setup, are in the README. The ones used most often:

```
python grand/selftest.py
python experiments/run_grand.py --C 16 --snrs 2,4,5,6,7,8,10 --code crc:0x9eb2:128:112
python experiments/plot_llr.py --C 32 --limit 8 --snrs 0,4,8,12 --conditions awgn,awgn_over3,
    awgn_under3,fade8,fade8_h0.1,fade1 --budget 100000 --out results/figures
python train_bitsemcom.py --C 32 --train_dir datasets/DIV2K_train_HR --val_dir datasets/
    kodak_patches256 --steps 30000 --out runs/C32_bench
python train_bitsemcom.py --C 32 --train_dir datasets/DIV2K_train_HR --val_dir datasets/
    kodak_patches256 --steps 30000 --grand crc:0x9eb2:128:112 --grand_budget 2000 --out runs/
    C32_grand
python experiments/evaluate.py --C 32 --ckpt bench=runs/C32_bench/best.model --ckpt grand=runs/
    C32_grand/best.model --snrs 0,2,4,6,8,10,12 --conditions awgn,fade8 --paths noiseless,
```

```
    uncoded_soft,uncoded_soft_il,uncoded_hard,coded_soft --budget 2000 --out results/eval_C32.
    jsonl
python experiments/plot_results.py results/eval_C32.jsonl --out results/figures
python experiments/plot_review_gains.py results/eval_C32.jsonl --b1e5 results/eval_C32_b1e5.jsonl
     --out docs/figures/grand_review
```

# 10 Glossary

**Semantic bits**
The bits the bit mapper produces; the network decides what they mean.

**Token**
One of the 256 feature vectors the encoder produces for a $256 \times 256$ image; each becomes $C$ bits.

**LLR**
Log-likelihood ratio of one bit: how sure the receiver is, and in which direction (positive = bit 0).

**Soft value / $P(1)$**
The probability that a bit is 1, computed from the LLR. "Soft" receivers keep it; "hard" receivers round it to 0 or 1.

**Systematic code**
A code whose codeword contains the data bits unchanged, followed by parity bits.

**GRAND / ORBGRAND**
Decoding by guessing the noise pattern, most likely first, and checking each guess against the code. ORBGRAND orders the guesses by the bit reliabilities.

**Guess budget**
The maximum number of guesses per block before the decoder gives up.

**Abandoned block**
A block where the budget ran out; passed on as soft values (harmless).

**Miscorrected block**
A block where a wrong codeword passed the check; passed on as confidently wrong bits (harmful).

**Interleaver**
A fixed shuffle of bit positions that spreads a burst of errors over many codewords and tokens.

**Block fading**
A channel whose gain is constant over a block of symbols and changes between blocks.

**CSI** Channel state information: what the receiver believes about the SNR and the channel gain.

**Paired difference**
The difference between two receivers on the same image with the same bits and noise, averaged over images.